

Final Project – Mambo Parrot



Azhari Abbas, Natasha Gaertner, and Edmund Naras

Department of Engineering, Embry-Riddle Aeronautical University

AE 443 – 08DB – Experimental Dynamics & Control Lab

Professor: Bernardo Restrepo-Torres

GTA: John-David Adams

April 22, 2026

1 Introduction

The Mambo Parrot drone is a quadcopter whose motion is defined by both translational and rotational dynamics. In the inertial frame, the position of the drone is given by x , y , and z , while its orientation is described using the Euler angles ϕ , θ , and ψ , corresponding to roll, pitch, and yaw. These rotations are generated by moments produced from the propellers, and the rotational motion is governed by Euler's equations.

For a quadcopter, controlling the attitude is essential because orientation directly affects translation. Any change in roll or pitch alters the direction of thrust, which results in motion in the horizontal plane. Because of this coupling, the system cannot maintain stability or track a desired input without feedback control. In open loop, the system does not behave in a controlled or predictable way.

The objective of this experiment was to analyze the quadcopter response using both nonlinear and linear models, reduce the system to the attitude states, derive transfer functions for roll and pitch, and design a controller to improve stability and tracking performance. The simulation results were used to evaluate system behavior and the effectiveness of the designed controller.

2 Procedure

2.1 Simulation Procedure

The simulation was performed using the provided MATLAB and Simulink quadcopter model. The model was configured so that attitude commands could be applied directly by loading the reference signal file containing roll, pitch, and yaw inputs. The signal type was adjusted so that the simulation used orientation inputs instead of position inputs.

The nonlinear model was evaluated first. The simulation was run with data logging enabled so that the output orientation angles and reference signals were stored in the workspace. These results were then plotted and compared to evaluate how well the system followed the commanded inputs.

The model was then switched to the linear airframe representation, and the simulation was repeated using the same inputs. The linear response was plotted and compared to the

nonlinear case to observe the effect of linearization.

After the simulations, the state-space matrices were extracted and reduced to include only the attitude states and angular rates. These matrices were used to derive transfer functions for roll and pitch, which were then used to analyze the system in open loop.

A proportional-derivative controller was designed by matching the closed-loop characteristic equation to a standard second-order form using the desired natural frequency and damping ratio. The calculated gains were implemented in the model, and the simulation was run again to evaluate the controlled response.

2.2 Implementation Procedure

The implementation was performed by deploying the controller from the Simulink model directly onto the Mambo Parrot drone. The flight control system block was opened as a standalone model so that the controller could be compiled and uploaded to the hardware.

The drone was powered on and connected to the computer through a Bluetooth connection. After establishing communication, the model was built and deployed using the hardware interface in Simulink. The system was first initialized at a low power setting to verify that the motors responded correctly and that the connection was stable.

Once proper operation was confirmed, the power level was increased and the drone was commanded to execute the predefined maneuver. During the test, the system recorded the state variables and reference inputs, which were saved for post-processing.

After the test, the recorded data was plotted to compare the measured roll and pitch angles with the reference signals. These results were used to evaluate the performance of the controller in a real environment and to compare it with the simulation results.

3 Results and Analysis

3.1 Nonlinear Model Response

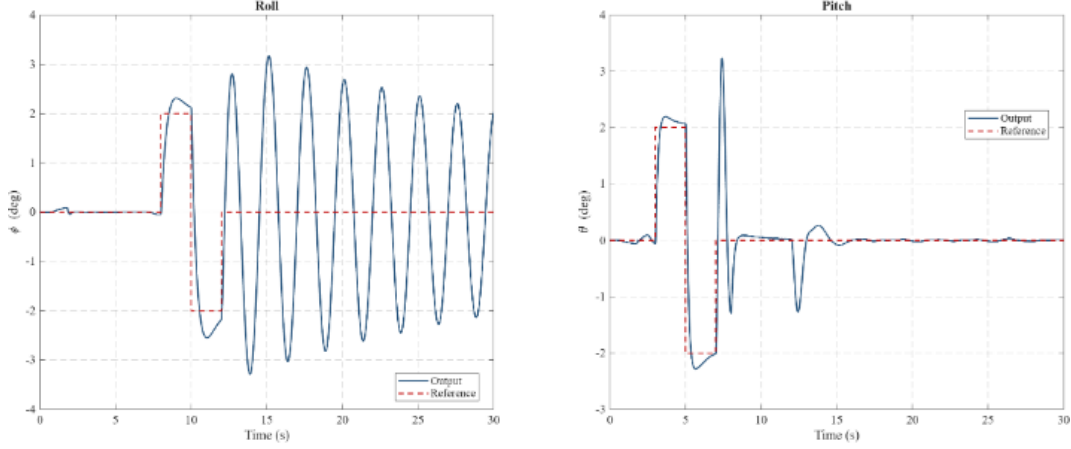


Figure 1: Nonlinear simulation response of roll (ϕ) and pitch (θ) angles compared to their reference inputs.

Figure 1 shows the nonlinear response of the system for both ϕ and θ compared to their reference inputs. The actual response does not follow the reference especially well, particularly during the attitude changes. The output shows noticeable oscillation and takes longer to settle, which indicates poor tracking performance.

This is expected for the nonlinear model in open loop. Since no controller is applied, the system has no way to correct for deviations from the reference. The nonlinear model also includes coupling effects and higher-order behavior that make the response less predictable and more oscillatory than the linear case.

Overall, the nonlinear response shows that the quadcopter does not have acceptable stability or tracking capability without control.

3.2 Linear Model Response

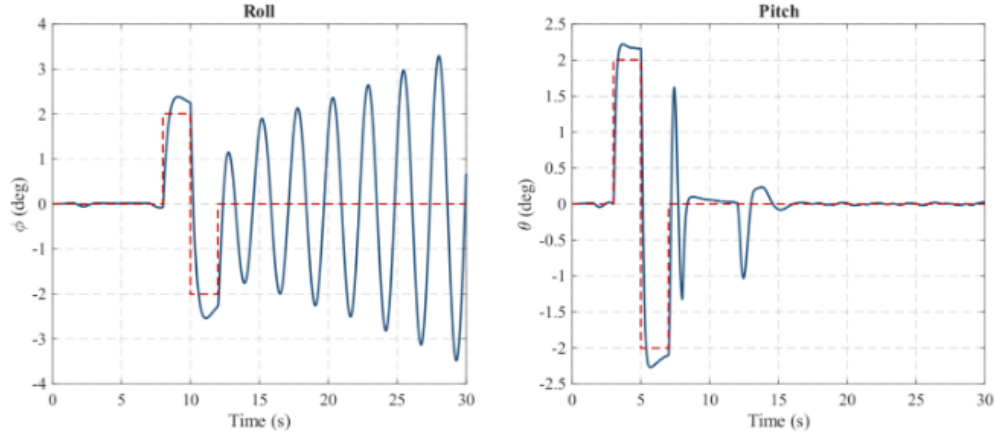


Figure 2: Linear simulation response of roll (ϕ) and pitch (θ) angles compared to their reference input.

Figure 2 shows the linear model response for ϕ and θ . Compared to the nonlinear model, the response is smoother and easier to interpret, but it still does not track the reference well. Oscillations are still present, and the system does not settle quickly to the commanded values.

The smoother behavior is due to the small-angle assumptions used in the linearized model, which remove some of the nonlinear coupling effects. Even so, the open-loop linear system still lacks the feedback needed to correct errors, so the overall tracking remains poor.

This confirms that while the linear model is easier to analyze, it still requires a controller to produce acceptable performance.

3.3 Reduced State-Space Model

The reduced state vector is

$$x = \begin{bmatrix} \phi \\ \theta \\ \psi \\ p \\ q \\ r \end{bmatrix} \quad (1)$$

Using the reduced-order attitude model, the state-space matrices are

$$A_{red} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -2.171528 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1.619205 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (2)$$

$$B_{red} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 642.893754 & 0 \\ 0 & 479.375125 \\ 0 & 0 \end{bmatrix} \quad (3)$$

$$C_{red} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

$$D_{red} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \quad (5)$$

The matrix A_{red} describes the attitude dynamics of the system, including the relationship between angular position and angular rate as well as the damping terms in roll and pitch. The matrix B_{red} shows how the control inputs affect the roll and pitch accelerations. The matrix C_{red} is the identity matrix, meaning the states are directly used as outputs, and D_{red} is zero, which indicates there is no direct feedthrough from input to output.

3.4 Transfer Function Derivation

From the reduced state-space model, the roll and pitch dynamics can be written directly from the state equations.

For roll,

$$\dot{\phi} = p \quad (6)$$

$$\dot{p} = -2.171528 p + 642.893754 u_{roll} \quad (7)$$

Since

$$p = \dot{\phi} \quad (8)$$

then

$$\dot{p} = \ddot{\phi} \quad (9)$$

Substituting into the second equation gives

$$\ddot{\phi} = -2.171528 \dot{\phi} + 642.893754 u_{roll} \quad (10)$$

Rearranging,

$$\ddot{\phi} + 2.171528 \dot{\phi} = 642.893754 u_{roll} \quad (11)$$

Taking the Laplace transform,

$$s^2 \Phi(s) + 2.171528 s \Phi(s) = 642.893754 U_{roll}(s) \quad (12)$$

Factoring out $\Phi(s)$,

$$\Phi(s) (s^2 + 2.171528 s) = 642.893754 U_{roll}(s) \quad (13)$$

Therefore, the roll transfer function is

$$G_{roll}(s) = \frac{\Phi(s)}{U_{roll}(s)} = \frac{642.893754}{s^2 + 2.171528 s} \quad (14)$$

For pitch,

$$\dot{\theta} = q \quad (15)$$

$$\dot{q} = -1.619205 q + 479.375125 u_{pitch} \quad (16)$$

Since

$$q = \dot{\theta} \quad (17)$$

then

$$\dot{q} = \ddot{\theta} \quad (18)$$

Substituting gives

$$\ddot{\theta} = -1.619205 \dot{\theta} + 479.375125 u_{pitch} \quad (19)$$

Rearranging,

$$\ddot{\theta} + 1.619205 \dot{\theta} = 479.375125 u_{pitch} \quad (20)$$

Taking the Laplace transform,

$$s^2 \Theta(s) + 1.619205 s \Theta(s) = 479.375125 U_{pitch}(s) \quad (21)$$

Factoring out $\Theta(s)$,

$$\Theta(s) (s^2 + 1.619205 s) = 479.375125 U_{pitch}(s) \quad (22)$$

Therefore, the pitch transfer function is

$$G_{pitch}(s) = \frac{\Theta(s)}{U_{pitch}(s)} = \frac{479.375125}{s^2 + 1.619205 s} \quad (23)$$

Both transfer functions contain a pole at the origin. This explains why the open-loop system does not settle properly and cannot track the reference input on its own.

3.5 Controller Design

A PD controller was selected for both roll and pitch, with the form

$$G_c(s) = K_p + K_d s \quad (24)$$

The desired closed-loop behavior was chosen to match a standard second-order system,

$$s^2 + 2\zeta\omega_n s + \omega_n^2 \quad (25)$$

Using

$$\omega_n = 2\pi \quad (26)$$

and

$$\zeta = 0.7 \quad (27)$$

gives

$$2\zeta\omega_n = 2(0.7)(2\pi) = 8.796459 \quad (28)$$

and

$$\omega_n^2 = (2\pi)^2 = 39.478418 \quad (29)$$

So the desired characteristic equation becomes

$$s^2 + 8.796459 s + 39.478418 \quad (30)$$

For roll, the open-loop plant is

$$G_{roll}(s) = \frac{642.893754}{s^2 + 2.171528 s} \quad (31)$$

Applying the PD controller gives the closed-loop denominator

$$s^2 + (2.171528 + 642.893754K_{d,roll}) s + 642.893754K_{p,roll} \quad (32)$$

Matching coefficients with the desired second-order equation,

$$2.171528 + 642.893754K_{d,roll} = 8.796459 \quad (33)$$

$$642.893754K_{p,roll} = 39.478418 \quad (34)$$

Solving for the gains,

$$K_{p,roll} = \frac{39.478418}{642.893754} = 0.061407 \quad (35)$$

$$K_{d,roll} = \frac{8.796459 - 2.171528}{642.893754} = 0.010305 \quad (36)$$

For pitch, the open-loop plant is

$$G_{pitch}(s) = \frac{479.375125}{s^2 + 1.619205 s} \quad (37)$$

The closed-loop denominator becomes

$$s^2 + (1.619205 + 479.375125K_{d,pitch}) s + 479.375125K_{p,pitch} \quad (38)$$

Matching coefficients,

$$1.619205 + 479.375125K_{d,pitch} = 8.796459 \quad (39)$$

$$479.375125K_{p,pitch} = 39.478418 \quad (40)$$

Solving gives

$$K_{p,pitch} = \frac{39.478418}{479.375125} = 0.082354 \quad (41)$$

$$K_{d,pitch} = \frac{8.796459 - 1.619205}{479.375125} = 0.014972 \quad (42)$$

These gains were then implemented in the simulation with the integral term set equal to zero.

3.6 Controlled System Response

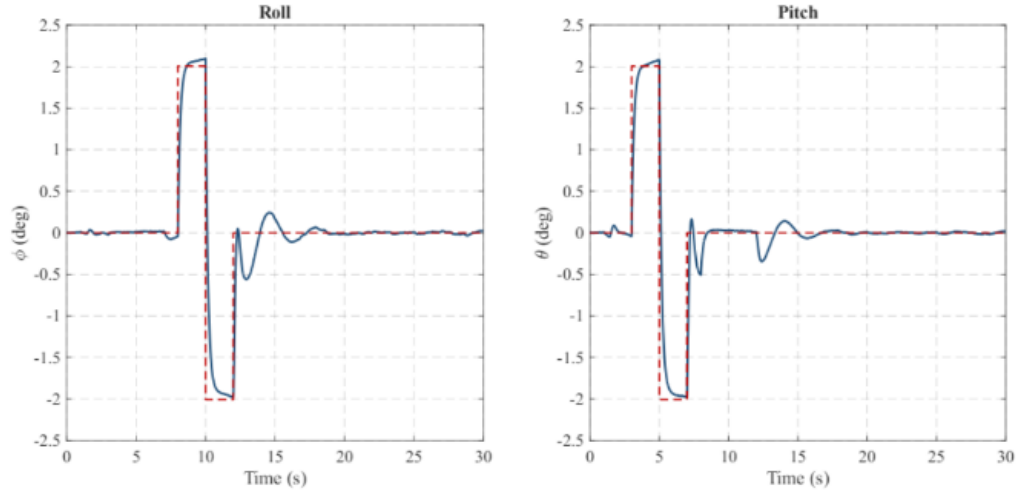


Figure 3: Closed-loop response of roll (ϕ) and pitch (θ) angles with the implemented PD controller.

Figure 3 shows the response after applying the PD controller. Compared to the open-loop cases, the system follows the reference much more closely and the oscillations are significantly reduced. The response settles faster and remains much closer to the commanded values.

The proportional term improves how strongly the system responds to the reference input, while the derivative term adds damping and reduces oscillation. Together, these gains produce a much more stable and usable response.

This improvement confirms that the controller design was successful and that the selected gains provided the desired closed-loop behavior.

3.7 Implementation Results

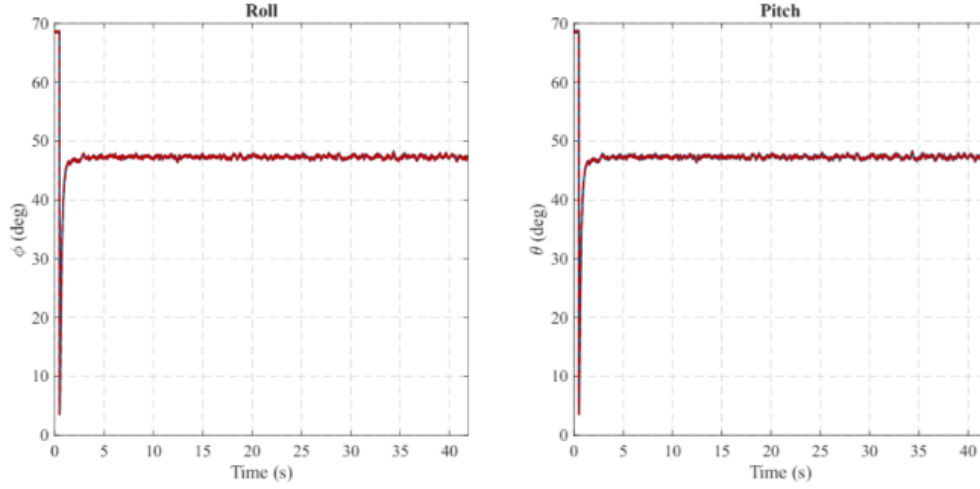


Figure 4: Experimental flight test response of roll (ϕ) and pitch (θ) compared to reference inputs.

The experimental results show that the system can generally track the reference inputs, although the response is noticeably noisier and less smooth than in simulation. The measured roll and pitch angles follow the overall trend of the reference signal, but small deviations and fluctuations are present throughout the maneuver.

The reference attitude signal appears noisy because it is generated by the outer position control loop rather than being a clean predefined input. As the quadcopter attempts to maintain position, small corrections in position translate into continuous adjustments in the desired attitude. Additionally, sensor noise from the onboard IMU and estimation errors further contribute to fluctuations in both the measured response and the reference signal. External disturbances such as air currents and motor variability also amplify this effect during flight testing.

Despite these effects, the controller is still able to maintain stability and track the general behavior of the reference input. However, the presence of noise and small tracking errors indicates that further tuning could improve performance. Increasing derivative filtering or refining gain values could help reduce sensitivity to noise, while implementing a full PID controller may improve steady-state accuracy.

4 Conclusion

This experiment demonstrated the importance of control in stabilizing a quadcopter system. Both the nonlinear and linear simulations showed that the open-loop system does not provide sufficient tracking or stability. By reducing the system to a state-space representation and deriving transfer functions, the behavior of the system was better understood and used to guide controller design.

A proportional-derivative controller was designed using a second-order approximation and implemented in simulation. The controller significantly improved system performance by reducing oscillations and increasing the ability of the system to track the reference input. The addition of derivative action improved damping, while the proportional term increased responsiveness.

There are several sources of possible error that would affect the results, especially during real implementation. Sensor noise and limited resolution can introduce fluctuations in the measured angles. Communication delays between the controller and the drone can affect timing and response accuracy. External disturbances such as air currents or slight changes in mass distribution can also impact performance. Additionally, the linear model used for controller design assumes small angles and does not capture all nonlinear effects, which can lead to differences between simulation and real behavior.

There are also several areas where the system could be improved. Further tuning of the controller gains could improve tracking accuracy and reduce steady-state error. Implementing a full PID controller instead of a PD controller could help eliminate steady-state offsets. More advanced control strategies, such as state feedback or adaptive control, could better handle nonlinear effects and disturbances. Improving sensor filtering and data processing would also reduce noise and improve overall system performance.

Overall, the simulation results show that the designed controller is effective in stabilizing the system and improving tracking. With additional refinement and consideration of real-world effects, the system performance could be further improved for practical implementation.